

Polynomial Regression & Regularization Master Guide (Telugu Transliteration)

Ee document lo, "PolynomialandRegularizationNotes.pdf" lo unna concepts ni base chesukuni, Non-linear data ni handle chese **Polynomial Regression** mariyu model overfitting ni control chese **Regularization (Ridge & Lasso)** gurinchi pin-to-pin Telugu transliteration lo (English letters tho) chala detailed ga explain chestanu. Meeru nidra lo lechina error lekunda code rayagalige antha strong coding practice, math, mariyu visualizations ikkada include chesanu.

1. Polynomial Regression Parichayam (Straight Line Kaadu, Curve Kaavali!)

Polynomial Regression ante enti?

Linear Regression lo manam oka straight line ($Y = mX + b$) ni fit chestam. Kani real life lo data eppudoo straight line lo undadu. Data curves lo, waves lo, leda s-shapes lo untundi. Appudu **Polynomial Regression** vadataru.

- Definition:** Input variable (X) mariyu output (Y) madhya unna non-linear (curved) relationship ni fit chese supervised learning method.

Nija Jeevitham lo Upayogalu (Real-Life Applications)

- Population Growth:** Time tho patu population growth exponential ga peragatam.
- Salary vs Experience:** Experience perigina koddi salary starting lo slow ga, tarwata fast ga peragatam.
- Electricity Consumption:** Temperature perigina koddi electricity usage curve la peragatam.

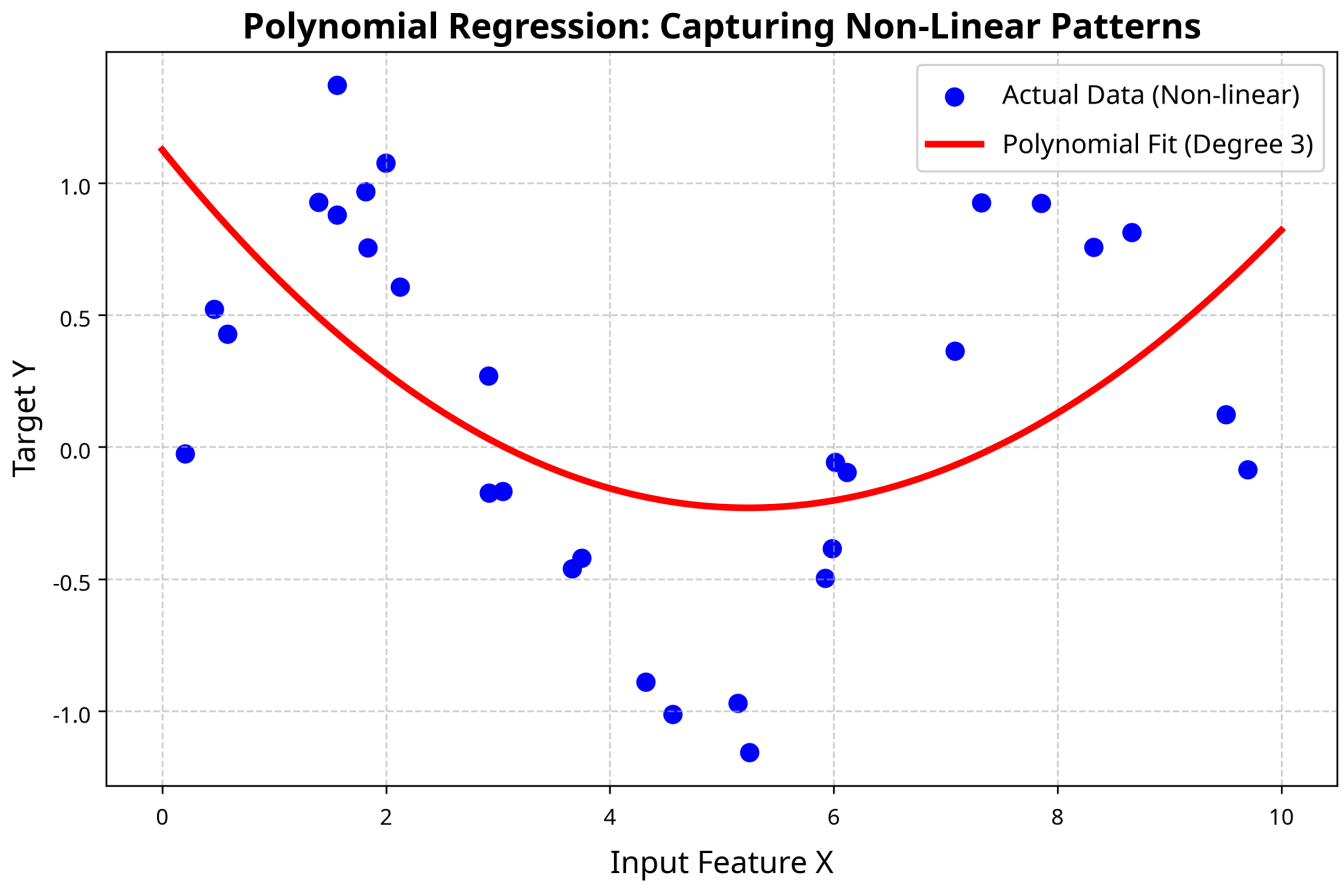
Polynomial Regression Equation

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \dots + \beta_n X^n$$

- Degree 1** → Linear Regression ($Y = mX + b$)
- Degree 2** → Quadratic Curve ($Y = a + bX + cX^2$)
- Degree 3** → Cubic Curve ($Y = a + bX + cX^2 + dX^3$)

Visualization: Polynomial Fit Curve

Kinda unna plot lo non-linear data ki polynomial regression curve ela fit avtundo chudandi:



2. Overfitting Problem (Too Complex Model)

Polynomial degree chala pedda ga pettam anukundam (e.g., Degree 10), appudu model training data lo unna prathi point ni (noise tho saha) memorizes chestundi.

- **Training Data:** 100% accuracy istundi.
- **Test Data / New Data:** Chala bad predictions istundi (Model generalise cheyalekapoyindi). Ee problem ne **Overfitting** antaru.

Overfitting ni ela tagginchali?

1. Cross-Validation vadatam.
2. **Regularization (Ridge & Lasso)** vadatam.

3. Regularization (Complexity Control & Overfitting Prevention)

Regularization ante enti?

Regularization ante machine learning model weights (coefficients) ki oka penalty add chesi, model ni simple ga unche technique. Idi model overfitting ni tagginchi, test data meeda manchi performance ichela chestundi.

Model training lo loss function ki oka penalty term add chestam: $\text{Loss} = \text{MSE} + \text{Penalty}$

Regularization Types: Ridge vs Lasso

Feature	Ridge Regression (L2)	Lasso Regression (L1)
Penalty Type	Squared coefficients ($\sum \beta^2$)	Absolute coefficients ($\sum \ \beta\ $)
Effect on Weights	Weights ni chala chinnaga chestundi, kani eppatiki zero cheyadu.	Sila weights ni exact ga 0 chestundi (Feature Selection).
Multicollinearity	Chala manchiga handle chestundi.	Features chala ekkuva unte oka feature ni matrame unchukuni migilina remove chestundi.

4. MASTER CODING SECTION: Step-by-Step Python Implementation

Meeru error lekunda code rayadaniki, scikit-learn use chesi Polynomial Regression, Ridge, mariyu Lasso models train chese complete Python code ikkada ichchanu.

```

# =====
# STEP 1: Import Required Libraries
# =====
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score

# =====
# STEP 2: Create Non-Linear Dataset
# =====
np.random.seed(42)
X = np.sort(np.random.rand(100, 1) * 10, axis=0)
# Y is a sine wave with some noise
y = np.sin(X).ravel() + np.random.normal(0, 0.2, 100)

# =====
# STEP 3: Train-Test Split
# =====
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# =====
# STEP 4: Polynomial Regression (Degree = 3)
# =====
# Features in polynomial form look like transform chestnut (X, X^2, X^3)
poly = PolynomialFeatures(degree=3)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

# Train Linear Regression on polynomial features
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)

y_pred_poly = poly_model.predict(X_test_poly)

print("--- POLYNOMIAL REGRESSION ---")
print(f"R2 Score: {r2_score(y_test, y_pred_poly):.4f}")
print(f"MSE: {mean_squared_error(y_test, y_pred_poly):.4f}")

# =====
# STEP 5: Ridge Regression (L2 Regularization)

```

```
# =====
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train_poly, y_train)
y_pred_ridge = ridge_model.predict(X_test_poly)

print("\n--- RIDGE REGRESSION (L2) ---")
print(f"R2 Score: {r2_score(y_test, y_pred_ridge):.4f}")

# =====
# STEP 6: Lasso Regression (L1 Regularization)
# =====
lasso_model = Lasso(alpha=0.01)
lasso_model.fit(X_train_poly, y_train)
y_pred_lasso = lasso_model.predict(X_test_poly)

print("\n--- LASSO REGRESSION (L1) ---")
print(f"R2 Score: {r2_score(y_test, y_pred_lasso):.4f}")
```

5. Real-Life & Data Science Applications

- **House Price Prediction:** Square feet mariyu location features ki non-linear pricing effects ni capture cheyadaniki Polynomial Regression vadataru.
- **Genetics & Biology:** Gene expression levels mariyu disease progression madhya non-linear relationships ni model cheyadaniki.
- **High-Dimensional Datasets:** Chala features (e.g., thousands of genes or financial indicators) unnapudu Ridge mariyu Lasso regression overfitting ni aapi best model ni istayi.

AI/ML/DL Model Recommendations

Polynomial features tho linear models vadadam baddhakam unna chotla manchi result istundi. Deep Learning lo kuda non-linear relationships ni handle cheyadaniki **Non-linear Activation Functions (ReLU, Sigmoid, Tanh)** mariyu **Weight Decay (L2 Regularization in Neural Networks)** vadataru.

References:

1. [PolynomialandRegularizationNotes.pdf](#) - Original PDF document for Polynomial Regression and Regularization (Ridge & Lasso).